

实验报告

动态规划算法

截止提交日期：2025 年 5 月 30 日

学号：_袁责铭_ 姓名：_2024012456_

一、设计目的

- 理解动态规划算法的基本思想，掌握将复杂问题分解为若干相互关联子问题的方法。
- 掌握动态规划中状态定义、状态转移方程和初始条件的设计方法。
- 能够结合实际问题建立动态规划模型，并通过程序实现和测试验证算法正确性。

二、设计内容

某校园外卖配送员需要从食堂出发，将餐品送到学生宿舍。校园道路抽象为一个 $m \times n$ 的网格，每个格子表示经过该位置所需的时间或体力消耗。配送员从左上角 $(0, 0)$ 出发，到达右下角 $(m-1, n-1)$ ，每次只能向右或向下移动一步。

请设计动态规划算法，求从起点到终点的最小总消耗，并输出一条对应的最优配送路径。

示例输入矩阵（ 4×4 ）：

1	3	5	9
8	1	3	4
5	0	6	1
8	8	4	0

从左上角到右下角的最优路径总消耗为 12，对应路径为

$(0, 0) \rightarrow (0, 1) \rightarrow (1, 1) \rightarrow (2, 1) \rightarrow (2, 2) \rightarrow (2, 3) \rightarrow (3, 3)$ 。

三、算法设计方案

3.1 状态定义

设 $dp[i][j]$ 表示从起点 $(0, 0)$ 出发，到达位置 (i, j) 所需的最小累计消耗。其中 $0 \leq i \leq m-1, 0 \leq j \leq n-1$ 。最终答案即为 $dp[m-1][n-1]$ 。

3.2 状态转移方程

由于每次只能向右或向下移动，位置 (i, j) 只能由 $(i-1, j)$ （从上方）或 $(i, j-1)$ （从左方）到达。取两者较小值加上当前格消耗，得到状态转移方程：

$$dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1]) \quad (i \geq 1, j \geq 1)$$

3.3 边界条件

- ◆ 起点： $dp[0][0] = grid[0][0]$ ，即起始位置的消耗就是该格子的值。
- ◆ 第一行（ $i=0$ ）：只能从左方到达， $dp[0][j] = dp[0][j-1] + grid[0][j]$ （ $j \geq 1$ ）。
- ◆ 第一列（ $j=0$ ）：只能从上方到达， $dp[i][0] = dp[i-1][0] + grid[i][0]$ （ $i \geq 1$ ）。

3.4 最终结果

按行从上到下、从左到右依次填表，最终 $dp[m-1][n-1]$ 即为所求最小总消耗。以示例矩阵为例，填表过程如下（黄色底色为终点最优值）：

	j=0	j=1	j=2	j=3
i=0	1	4	9	18
i=1	9	5	8	12
i=2	14	5	11	12
i=3	22	13	15	12

$dp[3][3] = 12$ ，与题目预期一致。

3.5 最优路径的记录与输出

从终点 $(m-1, n-1)$ 开始向起点回溯，每步比较 $dp[i-1][j]$ 与 $dp[i][j-1]$ 的大小，选择较小者所在方向作为上一步来源，直至到达 $(0, 0)$ ，然后将路径逆序输出。若两者相等，则任选其一（路径可能不唯一）。

四、动态规划适用性分析

- ◆ 可分解性：从 $(0, 0)$ 到 $(m-1, n-1)$ 的最优路径问题，可以分解为到达其上方格 $(m-2, n-1)$ 或左方格 $(m-1, n-2)$ 的最优路径问题，规模逐步缩小，直至到达边界。
- ◆ 重叠子问题：不同路径在到达某个中间格 (i, j) 时会汇聚，若使用暴力枚举， (i, j) 的最小消耗会被重复计算。动态规划通过填表将每个子问题只计算一次，避免了冗余。
- ◆ 最优子结构：到达 (i, j) 的最优路径必然包含到达其前驱节点的最优路径——这正是最优子结构性质。因此可以通过子问题的最优解推导出原问题的最优解。
- ◆ 效率优势：暴力枚举所有路径的数量为 $C(m+n-2, m-1)$ ，在 $m=n=20$ 时已超过十亿条，而动态规划的时间复杂度仅为 $O(m \times n)$ ，空间复杂度 $O(m \times n)$ ，具有绝对的效率优势。

五、核心代码实现

使用 Python 语言实现，包含状态初始化、边界处理、状态转移及路径回溯四个部分，代码结构清晰，含详细注释。

► 完整代码如下：

```
# 动态规划求网格最小消耗路径
```

```

def min_cost_path(grid):
    m, n = len(grid), len(grid[0])

    # dp[i][j] 表示从 (0,0) 到 (i,j) 的最小累计消耗
    dp = [[0]*n for _ in range(m)]

    # — 边界条件 —————
    dp[0][0] = grid[0][0]
    # 第一行: 只能一路向右
    for j in range(1, n):
        dp[0][j] = dp[0][j-1] + grid[0][j]
    # 第一列: 只能一路向下
    for i in range(1, m):
        dp[i][0] = dp[i-1][0] + grid[i][0]

    # — 状态转移 —————
    for i in range(1, m):
        for j in range(1, n):
            # 从上方或左方到达, 取较小值再加当前格消耗
            dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])

    # — 路径回溯 —————
    path = []
    i, j = m-1, n-1
    while i > 0 or j > 0:
        path.append((i, j))
        if i == 0:                # 只能来自左边
            j -= 1
        elif j == 0:              # 只能来自上边
            i -= 1
        elif dp[i-1][j] < dp[i][j-1]:
            i -= 1                # 上方更优
        else:
            j -= 1                # 左方更优
    path.append((0, 0))
    path.reverse()

    return dp[m-1][n-1], path

if __name__ == '__main__':
    m = int(input('请输入行数 m: '))
    n = int(input('请输入列数 n: '))
    print('请逐行输入网格消耗值 (空格分隔): ')
    grid = [list(map(int, input().split())) for _ in range(m)]

    cost, path = min_cost_path(grid)

```

```
path_str = '->'.join(f'({r},{c})' for r,c in path)
print(f'最小总消耗: {cost}')
print(f'最优路径: {path_str}')
```

代码说明：路径回溯从终点出发，依据 dp 表中上方与左方的大小关系，逐步追溯到起点，最终反转得到正向路径。

六、测试与结果分析

6.1 测试结果汇总

选取 4 组测试数据，覆盖方形矩阵、非方形矩阵及极小规模矩阵，测试结果如下：

测试编号	输入矩阵	最小总消耗	一条最优路径
1	[[1, 3, 5, 9], [8, 1, 3, 4], [5, 0, 6, 1], [8, 8, 4, 0]]	12	(0, 0)→(0, 1)→(1, 1)→(2, 1)→(2, 2)→(2, 3)→(3, 3)
2	[[1, 2, 3], [4, 5, 6]]	12	(0, 0)→(0, 1)→(0, 2)→(1, 2)
3	[[5, 9], [1, 2]]	8	(0, 0)→(1, 0)→(1, 1)
4	[[2, 1, 3], [6, 2, 4], [3, 5, 1]]	10	(0, 0)→(0, 1)→(1, 1)→(1, 2)→(2, 2)

6.2 逐组结果分析

- ◆ 测试 1（4×4 矩阵）：最小消耗 12，程序输出路径 (0, 0)→(0, 1)→(1, 1)→(2, 1)→(2, 2)→(2, 3)→(3, 3)，与题目示例一致，结果正确。该路径巧妙绕开了高消耗格子，充分体现了动态规划的全局最优性。
- ◆ 测试 2（2×3 矩阵）：最小消耗 12，最优路径沿第一行走到底再向下，程序输出 (0, 0)→(0, 1)→(0, 2)→(1, 2)，验证了非方形矩阵的处理正确性。
- ◆ 测试 3（2×2 矩阵）：最小消耗 8，路径为 (0, 0)→(1, 0)→(1, 1)。此例说明向下走再向右，比先向右再向下（5+9+2=16 vs 5+1+2=8）消耗更少，算法正确地选择了最优方向。

◆ 测试 4 (3×3 矩阵)：最小消耗 10，路径

(0, 0)→(0, 1)→(1, 1)→(1, 2)→(2, 2)，程序输出与预期相符。验证了路径不唯一情况下算法仍能输出合法且最优的路径。

七、总结与讨论

7.1 对动态规划思想的理解

动态规划的核心是「记住已经解决的子问题，避免重复计算」。与分治算法不同，动态规划要求子问题之间存在重叠，且整体最优解可由子问题最优解推导而来（即最优子结构）。本实验中，每个格子的最优到达消耗只依赖于其上方和左方两个格子，子问题关系清晰，非常适合用填表法（自底向上）实现。

7.2 状态定义与转移方程设计的体会

状态定义是动态规划设计的核心。本题中，将 $dp[i][j]$ 定义为到达 (i, j) 的最小消耗，既能准确描述子问题，又便于从中推导出最终答案。状态转移方程 $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$ 直接来源于移动规则，逻辑清晰。边界条件的设置（第一行和第一列的特殊处理）也至关重要，稍有疏漏便会导致结果错误。

7.3 与暴力枚举相比的优势

暴力枚举所有路径的数量随网格规模呈组合爆炸式增长： $m=n=10$ 时约有 48620 条路径， $m=n=20$ 时超过 1.7 亿条。而动态规划只需 $O(m \times n)$ 的时间和空间，对 $m=n=1000$ 的大规模网格也能在毫秒内给出答案，效率提升极为显著。

7.4 实验过程中遇到的问题及解决方法

在路径回溯阶段，最初未考虑 $i=0$ 或 $j=0$ 时只能单向回溯的情况，导致越界错误。解决方法是在回溯时增加边界判断：当 $i=0$ 时只能向左 ($j--$)，当 $j=0$ 时只能向上 ($i--$)，仅在两者均大于 0 时才比较 dp 值选择方向，从而保证回溯过程不越界。

7.5 加入障碍物时的算法修改思路

若网格中存在障碍物（即某些位置不可通过），只需在状态转移时增加一个判断：若当前格子 (i, j) 是障碍物，则设 $dp[i][j] = +\infty$ （或一个极大值），表示

无法到达该位置。具体修改如下：

- ◆ 若 $\text{grid}[i][j] = -1$ （代表障碍物），令 $\text{dp}[i][j] = +\text{inf}$ ，跳过该格子。
- ◆ 转移时，若 $\text{dp}[i-1][j]$ 或 $\text{dp}[i][j-1]$ 为 $+\text{inf}$ ，则视为不可达，只取另一方向的值。
- ◆ 若终点 $\text{dp}[m-1][n-1]$ 仍为 $+\text{inf}$ ，说明不存在合法路径，输出「无法到达」。

此改动仅需在原代码基础上添加数行条件判断，整体框架不变，体现了动态规划良好的可扩展性。